

Rheodemy Backend Build Plan (Hackathon MVP)

Overview

Rheodemy is a real-time, pay-as-you-learn educational marketplace powered by the Interledger Protocol (ILP) using Rafiki. Students pay teachers per minute during live sessions using streaming micropayments. Payments start and stop instantly based on session activity and user-defined spending limits. The system uses custodial wallets and abstracts Interledger from the end-user experience to ensure frictionless onboarding.

1. MVP Scope & Demo Constraints (Strictly for Demo Day)

To eliminate points of failure on stage, the MVP adheres to these pragmatic engineering guidelines:

- **Direct Wallet Streaming:** To avoid complex multi-step escrow settlement logic under hackathon deadlines, payments stream directly from the Student Wallet to the Teacher Wallet. The platform's cut is calculated conceptually on the UI/logs.
- **The 2–5 Second Ticker:** The "Chunky Ticker" operates on a snappy 2-to-5 second interval. This ensures the judges see numbers moving dynamically during a short pitch without exhausting database connections.
- **Instant Stop Override:** When a session ends, the backend immediately halts the payment stream, completely overriding the active ticker interval.
- **The Kill Switch Safety Net:** A dedicated admin endpoint (`/api/session/:id/force-kill`) is built to manually terminate a session and freeze payment loops if a frontend client crashes or network connectivity drops on stage.
- **Hardcoded FX Rate:** No external currency API connections. A static, reliable exchange rate configuration (e.g., `const USD_TO_NGN = 1500;`) is hardcoded to show cross-currency calculations flawlessly.
- **Restricted WebSocket Scope:** Socket.io is used exclusively for signaling session states (start/stop) and transmitting live UI timer/cost data. Actual ILP ledger operations remain securely inside the core backend payment service.

2. Core Features Being Built

1. **Authentication & Onboarding:** Single unified signup/login. New accounts begin in an `UNASSIGNED` role state, funneling users to a post-registration screen to choose their role (Student or Teacher).
2. **Custodial Wallet System:** Automatically instantiates a local wallet profile linked to an ILP testnet wallet address (via Rafiki) upon registration, loaded with demo/test funds.

3. **Teacher Marketplace:** Simple directory endpoints displaying active teachers, their rates per minute, and their live availability status.
4. **Live Session Signaling:** WebSocket infrastructure to manage live connection states (**ACTIVE**, **COMPLETED**, **PAUSED**).
5. **Ledger System:** A robust transactional ledger mapping completed sessions to final reconciled ILP payment hashes for auditable historical tracking.

3. Development Phases & Responsibilities

Phase 1 — Core Setup & Onboarding (Foundation)

- **Assigned to:** Laurretta
- **Tasks:**
 - Initialize Node.js repository with TypeScript/Express or NestJS.
 - Provision PostgreSQL database and integrate Prisma ORM.
 - Design the unified **User** and **Wallet** schemas (incorporating the **UNASSIGNED** role state).
 - Build secure JWT-based authentication endpoints.
 - Build the post-registration role selection endpoint.

Phase 2 — Marketplace & Teacher Profiles

- **Assigned to:** Laurretta
- **Tasks:**
 - Construct the **TeacherProfile** data model.
 - Develop marketplace endpoints (**GET /teachers**) with filters for availability and rates.
 - Build endpoints allowing teachers to toggling their active online status.

Phase 3 — WebSocket Session Signaling

- **Assigned to:** Laurretta
- **Tasks:**
 - Establish Socket.io infrastructure within the server backend.
 - Implement event listeners for **session:start** and **session:stop**.
 - Develop standard REST endpoints to systematically initialize and change **Session** record states in PostgreSQL.

Phase 4 — Rafiki & Payment Integration

- **Assigned to:** Olamide
- **Tasks:**
 - Set up a local Rafiki testnet simulator instance.
 - Provision test networks for Student and Teacher wallet endpoints.
 - Write the core Payment Streaming service using Open Payments APIs to generate grants, quotes, and outgoing payments.

Phase 5 — Chunky Ticker & Safety Logic

- **Assigned to:** Olamide
- **Tasks:**
 - Build the 2–5 second in-memory interval loop to track running micro-balances.
 - Implement user spending limit checks within the ticker loop to trigger automated stop triggers.
 - Build the `/api/session/:id/force-kill` manual admin endpoint to instantly freeze payment states.
 - Inject the static FX configuration parameters for seamless multi-currency backend processing.

Phase 6 — Ledger Reconciliations & Testing

- **Assigned to:** Both Developers
- **Tasks:**
 - Develop the transactional `Ledger` database entries to commit final session tallies.
 - Perform comprehensive end-to-end simulation tests via Postman (Session Start $\rightarrow$ Payment Streaming $\rightarrow$ Limit Triggered or Manual Kill Switch Triggered).